

Doc. No.: WG21/P0250R2
Date: 2016-07-10
Author: Hans-J. Boehm
Email: hboehm@google.com

Target audience: SG1 and CWG

P0250R2: Wording improvements for initialization and thread ids (CWG 2046, 1784)

This is an attempt to turn the Kona SG1 discussion of CWG 2046 into wording. This revision also reflects discussion in Jacksonville and Oulu.

Happens-before constraints on the implementation

In several places the standard needs to require that some operation, e.g. an initialization, happens before another one, in roughly, but not quite, the sense of the 1.10.1 memory model. The difference is that we expect this happens before relation to compose with other happens-before relationships. But our normal happens before relation no longer composes in this way, due to the presence of `memory_order_consume`. Hence we suggest introducing the required notion in 1.10.1.

There was some discussion in Jacksonville about using "synchronizes with" instead of introducing new terminology. That appears to be technically workable, but rather misleading. "Synchronizes with" is currently only defined between actions by different threads, and does not include sequencing. Many of the guarantees we would like to provide will often rely on sequencing rather than synchronization. Thus my current feeling is to introduce new terminology.

The current standard sometimes uses "sequenced before" to express this kind of relationship. This is incorrect and repaired below, since sequencing is a relation on evaluations within a single thread.

Parallel initialization for sequential programs

In Kona, SG1 rather quickly concluded that a C++ program should be allowed to construct multiple threads for constructors etc., even if the program itself did not use threads. That is not reflected here, mostly because it is a more profound issue than we appreciated at the time.

If we went this route, we would break compatibility with C++03, in what may be a rather significant way. Static constructors for old single-threaded programs would be required to be thread-safe. This is unlikely to be automatically true. It is not that uncommon for constructors to e.g. add to a global data structure. The order in which items are added may not matter. But such a data structure implemented in a single-threaded program is unlikely to be thread-safe.

In Jacksonville, it became clear that some of the confusion here stems from uncertainty about the motivation for the current wording. Currently, we very explicitly allow static constructors to run after the start of main, whether or not other threads are started. This appears to be motivated by the intent to support e.g. lazily loading a dynamic library when a function symbol is referenced, as with `RTLD_LAZY` on Posix systems. Even if static namespace-scope constructors are run immediately in library loading, the library may be implicitly loaded after the start of main. This seems inherently unsafe; if a constructor requiring a mutex M is run as part of lazy library loading at a point at which M is held by the loading thread, we unavoidably deadlock. And there seems to be no easy way to avoid that.

The solutions appear to be:

1. Avoid dynamic initialization of namespace-scope statics. Good advice, but probably not something we can dictate.
2. Prohibit this kind of asynchronous constructor execution. This would prohibit a technique that is useful for those of us who avoid such dynamic initialization anyway.
3. The current approach seems to work in practice (modulo other constructor ordering issues) because constructors generally run at reasonably well-defined points at which such deadlocks do not occur. We could pin down those points.

Here we follow the last path, but choose not to be specific, and instead make the precise points at which construction occurs implementation-defined. This seems to better reflect the actual status quo. It also means that users who avoid dynamically initialized static objects can continue to take advantage of facilities like `RTLD_LAZY`.

Running a constructor while main is running inherently implies some notion of concurrency, at least between constructors and main-line code. In the multi-threaded case, constructors run fully concurrently with whatever non-initializing threads happen to be running at the time. But it really only results in constructors running in existing threads, not the starting of additional threads solely to increase constructor parallelism. This seems to be a bit easier to specify than the original version, which allowed parallel constructor execution once a user thread had been started. SG1 generally feels that static namespace-scope constructors should be avoided, and thus speeding them up is a low priority; thus we decided to restrict such constructors to existing threads, which appears to be consistent with known implementations.

Thread id for main

We concluded in Kona that main did not necessarily have a thread id, and that some implementations went out of their way to keep it from having one. After looking at the N4567 wording again, my conclusion is that this interpretation of the standard is a stretch. Some of this wording may have improved since the implementers looked at it.

The first sentence of 1.10 [intro.multithread] states

A thread of execution (also known as a thread) is a single flow of control within a program, including the initial invocation of a specific top-level function, and recursively including every function invocation subsequently executed by the thread.

30.3.1.1 [thread.thread.id] states

An object of type `thread::id` provides a unique identifier for each thread of execution and a single distinct value for all thread objects that do not represent a thread of execution (30.3.1).

30.3.2 [thread.thread.this] states

```
thread::id this_thread::get_id() noexcept;
```

Returns: An object of type `thread::id` that uniquely identifies the current thread of execution. No other thread of execution shall have this id and this thread of execution shall always have this id. The object returned shall not compare equal to a default constructed `thread::id`.

I would nonetheless suggest changing the first sentence of 3.6.1 [basic.start.main] as follows:

A program shall contain a global function called `main`, which is the designated start of the program and of its initial thread of execution.

There were some voices in Jacksonville in favor of preparing the standard for execution agents that do not support a thread id (or thread local storage). This would simplify GPU execution, but it seems to require that the standard library specify which calls need thread local storage, and which do not, something we don't currently specify. It is unclear to me that we should address that issue in this context.

Wording changes for construction/destruction ordering

These are roughly relative to the C++17 CD.

Include the immediately preceding change to 3.6.1 [basic.start.main].

(Fixes an editorial ugliness that somehow crept in.) Move 1.9p6, which uses *sequenced before*, to someplace below the definition of *sequenced before* in 1.9p13 [intro.execution].

Add after 1.10.1p10 [intro.races]:

An evaluation A *strongly happens before* an evaluation B if either

- A is sequenced before B, or
- A synchronizes with B, or
- A strongly happens before X and X strongly happens before B.

[Note: In the absence of consume operations the *happens before* and *strongly happens before* relations are identical. *Strongly happens before* essentially excludes consume operations.]

There may be cleaner ways to introduce this.

Change the last part of 3.6.2p2 [basic.start.static] as follows:

If constant initialization is not performed, a variable with static storage duration (3.7.1) or thread storage duration (3.7.2) is zero-initialized (8.6). Together, zero-initialization and constant initialization are called static initialization; all other initialization is dynamic initialization. Within each phase, static initialization shall ~~be performed~~ strongly happen before (1.10.1) any dynamic initialization ~~takes place~~. [Note: The dynamic initialization of non-local variables is described in 3.6.3; that of local static variables is described in 6.7. -- end note]

Change the middle two bullet points in 3.6.3p2 [basic.start.dynamic] as follows:

- If V has partially-ordered initialization, W does not have unordered initialization, and V is defined before W in every translation unit in which W is defined, the initialization of V is sequenced before the initialization of W if the program does not start a thread (1.10) and otherwise strongly happens before the initialization of W.
- Otherwise, if a program starts a thread before either V or W is initialized, the initializations of V and W may occur in different threads, or are unsequenced in the same thread.

Change the beginning of 3.6.3p3 [basic.start.dynamic] as follows:

It is implementation-defined whether the dynamic initialization of a non-local non-inline variable with static storage duration ~~happens before~~ is sequenced before the first statement of main, in the same thread of execution as main, or can be deferred, and possibly occur in other user-created threads. In the latter case ~~If the initialization is deferred to happen after the first statement of main,~~ it strongly happens before the first odr-use (3.2) of any non-inline function or non-inline variable defined in the same translation unit as the variable to be initialized. The program points at which such dynamic initialization may occur are implementation-defined. [Note: Such points should be chosen to allow the programmer to avoid situations in which the constructor invocation occurs while a mutex is held, thus potentially causing deadlock. --end note]

Change 3.6.3p4 as follows:

It is implementation-defined whether the dynamic initialization of a non-local inline variable with static storage duration ~~happens before~~ is sequenced before the first statement of main, in the same thread of execution as main, or can be deferred, and possibly occur in other user-created threads. In the latter case ~~If the initialization is deferred to happen after the first statement of main,~~ it strongly happens before the first odr-use (3.2) of any non-inline function or non-inline variable defined in the same translation unit as the variable to be initialized. The program points at which such dynamic initialization may occur are implementation-defined.

Change 3.6.3p5 as follows to fix what looks like an unrelated bug, and pin down initialization of thread-locals:

It is implementation-defined whether the dynamic initialization of a non-local non-inline variable with ~~static or~~ thread storage duration is sequenced before the first statement of the initial function of the thread. If the initialization is deferred to some point in time sequenced after the first statement of the initial function of the thread, the initialization associated with the entity for thread ~~t~~ is sequenced before the first odr-use (3.2) by ~~t~~ of any variable with thread storage duration defined in the same translation unit as the variable to be initialized. Such initialization is sequenced immediately before such an odr-use. That is, it is sequenced before the odr-use, and every other evaluation that is sequenced before the odr-use is sequenced before the initialization.

The preceding had similar issues to static variables, in that it allowed essentially concurrent initialization of thread-locals. As far as we can tell implementations really initialize in response to first access. Thus we did not resort to implementation-defined behavior here.

Change 3.6.4p1 [basic.start.term] as follows:

Destructors (12.4) for initialized objects (that is, objects whose lifetime (3.8) has begun) with static storage duration, and functions registered with `at_exit`, are called either by the main thread of execution as a result of returning from main, ~~or and~~ as part of the call to `std::exit` (18.5). The return from main or the call to `std::exit`, respectively, is sequenced before the destructor and registered function invocations.

Destructors for initialized objects with thread storage duration within a given thread are called as a result of returning from the initial function of that thread and as a result of that thread calling `std::exit`. The completions of the destructors for all initialized objects with thread storage duration within that thread ~~are sequenced~~ strongly happen before the initiation of the destructors of any object with static storage duration. If the completion of the constructor or dynamic initialization of an object with thread storage duration is sequenced before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first.

If the completion of the constructor or dynamic initialization of an object with static storage duration ~~is sequenced~~ strongly happens before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first. ~~[Note: This definition permits concurrent destruction. -- end note]~~ If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. For an object of array or class type, all subobjects of that object are destroyed before any block-scope object with static storage duration initialized during the construction of the subobjects is destroyed. If the destruction of an object with static or thread storage duration exits via an exception, `std::terminate` is called (15.5.1).

Change 3.6.4p3 [basic.start.term] as follows:

If the completion of the initialization of an object with static storage duration ~~is sequenced~~ strongly happens before a call to `std::atexit` (see <cstdlib>, 18.5), the call to the function passed to `std::atexit` is sequenced before the call to the destructor for the object. If a call to `std::atexit` ~~is sequenced~~ strongly happens before the completion of the initialization of an object with static storage duration, the call to the destructor for the object is sequenced before the call to the function passed to `std::atexit`. If a call to `std::atexit` ~~is sequenced~~ strongly happens before another call to `std::atexit`, the call to the function passed to the second `std::atexit` call is sequenced before the call to the function passed to the first `std::atexit` call.

Change 6.7p4 [stmt.dcl] to

Dynamic initialization of a block-scope variable with static storage duration (3.7.1) or thread storage duration (3.7.2) is performed the first time control passes through its declaration; such a variable is considered initialized upon the completion of its initialization. If the initialization exits by throwing an exception, the initialization is not complete, so it will be tried again the next time control enters the declaration. If control enters the declaration concurrently while

the variable is being initialized, the concurrent execution shall wait for completion of the initialization. [92] If the declaration is sequenced before another evaluation E, then the initialization strongly happens before E. If control re-enters the declaration recursively while the variable is being initialized, the behavior is undefined. [Example: ...]